

Lab05 Kaleo256333

Kaleo Campos

2026-09-16

```
library(readr)
library(dplyr)
library(leaflet)
library(htmlwidgets)
library(webshot2)
```

Parte 1

Crie uma função que:

- Receba um valor de `TAIL_NUMBER` (por exemplo, N431WN);
- Produza uma tabela (tidy) com todos os trajetos realizados pela aeronave (ordenadas por data e hora, contendo todas as colunas do arquivo `flights.csv.zip`);
- Produza um mapa que apresente todo o trajeto voado pela aeronave ao longo de todo o ano; o trajeto deve ser apresentado de maneira linear no tempo;
- O mapa deve ser decorado com estatísticas do seu interesse.

A função se chama `analisa_aeronave` e recebe os argumentos `tail_number` (código identificador do avião) e `arquivo` (caminho para o arquivo de vôos, relativo ao diretório de trabalho). O arquivo é lido por partes, com `read_csv_chunked`, e o mapa usa as coordenadas exatas dos aeroportos, obtidas em `airports.csv`. O retorno é uma lista nomeada com a tabela usada no gráfico e o gráfico propriamente dito.

Especificação dos tipos de colunas

Como a leitura é feita em lotes, a especificação de tipos é declarada explicitamente. Isso evita que o `readr` adivinhe tipos diferentes em lotes diferentes (por exemplo, uma coluna quase toda faltante em um lote). Aqui é usado `cols` (e não `cols_only`), porque o enunciado pede que a tabela final contenha **todas** as colunas do arquivo de vôos.

```
tipos_voos <- cols(
  .default      = col_double(),
  AIRLINE       = col_character(),
  TAIL_NUMBER   = col_character(),
  ORIGIN_AIRPORT = col_character(),
  DESTINATION_AIRPORT = col_character(),
  CANCELLATION_REASON = col_character()
)
```

Funções auxiliares

`para_minutos` converte um horário no formato HHMM (por exemplo, 1435) para o número de minutos desde a meia-noite (nesse caso, 875). Essa conversão é usada apenas para ordenar os vôos dentro de um mesmo dia e para calcular a duração entre a partida programada e o pouso.

```
para_minutos <- function(horario_hhmm) {
  (horario_hhmm %/% 100) * 60 + (horario_hhmm %% 100)
}
```

`filtra_aeronave` é a função de callback usada na leitura por partes: para cada lote lido do arquivo, ela mantém apenas as linhas da aeronave de interesse. O argumento `pos` (posição do ponteiro dentro da base) não é usado internamente, mas precisa existir, caso contrário a leitura por partes não funciona.

```
filtra_aeronave <- function(input, pos, tail_number) {
  input %>%
    filter(TAIL_NUMBER == tail_number)
}
```

Função principal

```
analisa_aeronave <- function(tail_number, arquivo) {

  ## airports.csv é procurado na pasta do arquivo de vôos e, se não estiver lá, no diretório
  arquivo_aeroportos <- file.path(dirname(arquivo), "airports.csv")
  if (!file.exists(arquivo_aeroportos)) {
    arquivo_aeroportos <- "airports.csv"
  }
}
```

```

aeroportos <- read_csv(
  arquivo_aeroportos,
  col_types = cols_only(
    IATA_CODE = "c",
    AIRPORT   = "c",
    CITY      = "c",
    STATE     = "c",
    LATITUDE  = "d",
    LONGITUDE = "d"
  )
)

voos_da_aeronave <- read_csv_chunked(
  file      = arquivo,
  callback  = DataFrameCallback$new(
    function(input, pos) filtra_aeronave(input, pos, tail_number)
  ),
  chunk_size = 1e6,
  col_types  = tipos_voos
)

if (nrow(voos_da_aeronave) == 0) {
  stop("Nenhum voo encontrado para a aeronave ", tail_number, ".")
}

trajetos <- voos_da_aeronave %>%
  mutate(
    ## as colunas são lidas como double e sprintf exige inteiros
    DATA = as.Date(sprintf(
      "%04d-%02d-%02d", as.integer(YEAR), as.integer(MONTH), as.integer(DAY)
    )),
    PARTIDA_MIN = para_minutos(SCHEDULED_DEPARTURE)
  ) %>%
  arrange(DATA, PARTIDA_MIN) %>%
  left_join(
    aeroportos %>%
      select(
        IATA_CODE,
        AEROPORTO_ORIGEM = AIRPORT,
        CIDADE_ORIGEM    = CITY,
        LAT_ORIGEM       = LATITUDE,
        LON_ORIGEM       = LONGITUDE
      )
  )

```

```

    ),
    by = c("ORIGIN_AIRPORT" = "IATA_CODE")
  ) %>%
  left_join(
    aeroportos %>%
      select(
        IATA_CODE,
        AEROPORTO_DESTINO = AIRPORT,
        CIDADE_DESTINO    = CITY,
        LAT_DESTINO        = LATITUDE,
        LON_DESTINO        = LONGITUDE
      ),
    by = c("DESTINATION_AIRPORT" = "IATA_CODE")
  ) %>%
  mutate(
    VELOCIDADE_MEDIA = DISTANCE / (AIR_TIME / 60), # milhas por hora
    ORDEM             = row_number()             # posição do voo no ano
  )

## vãos cancelados não têm tempo de voo e alguns aeroportos aparecem codificados por número
trechos_no_mapa <- trajetos %>%
  filter(
    !is.na(LAT_ORIGEM), !is.na(LON_ORIGEM),
    !is.na(LAT_DESTINO), !is.na(LON_DESTINO),
    is.finite(VELOCIDADE_MEDIA)
  )

aeroportos_visitados <- trechos_no_mapa %>%
  group_by(
    CODIGO      = DESTINATION_AIRPORT,
    AEROPORTO   = AEROPORTO_DESTINO,
    CIDADE      = CIDADE_DESTINO,
    LATITUDE    = LAT_DESTINO,
    LONGITUDE   = LON_DESTINO
  ) %>%
  summarise(POUSOS = n(), .groups = "drop")

## cor da linha: ordem cronológica do voo; espessura: velocidade média
paleta_tempo <- colorNumeric(
  palette = c("#4575b4", "#ffffbf", "#d73027"),
  domain  = trechos_no_mapa$ORDEM
)

```

```

faixa_velocidade <- range(trechos_no_mapa$VELOCIDADE_MEDIA)
espessura <- 1 + 5 * (trechos_no_mapa$VELOCIDADE_MEDIA - faixa_velocidade[1]) /
  diff(faixa_velocidade) # reescala a velocidade para o intervalo [1, 6]

mapa <- leaflet() %>%
  addTiles() %>%
  addCircleMarkers(
    data      = aeroportos_visitados,
    lng       = ~LONGITUDE,
    lat       = ~LATITUDE,
    radius     = ~3 + 2 * sqrt(POUSOS),
    color      = "#333333",
    weight     = 1,
    fillColor  = "#ffffff",
    fillOpacity = 0.8,
    popup     = ~paste0(
      "<b>", CODIGO, "</b> - ", AEROPORTO, "<br/>",
      CIDADE, "<br/>",
      "Pousos da aeronave: ", POUSOS
    )
  )

## os trechos entram na ordem cronológica, um a um, para que o trajeto seja percorrido lin
for (i in seq_len(nrow(trechos_no_mapa))) {
  trecho <- trechos_no_mapa[i, ]
  mapa <- mapa %>%
    addPolylines(
      lng      = c(trecho$LON_ORIGEM, trecho$LON_DESTINO),
      lat      = c(trecho$LAT_ORIGEM, trecho$LAT_DESTINO),
      color     = paleta_tempo(trecho$ORDEM),
      weight    = espessura[i],
      opacity   = 0.6,
      popup     = paste0(
        "Vôo ", trecho$ORDEM, " de ", nrow(trechos_no_mapa), "<br/>",
        trecho$DATA, "<br/>",
        trecho$ORIGIN_AIRPORT, " -> ", trecho$DESTINATION_AIRPORT, "<br/>",
        "Distância: ", trecho$DISTANCE, " milhas<br/>",
        "Velocidade média: ", round(trecho$VELOCIDADE_MEDIA), " mph"
      )
    )
}

```

```

mapa <- mapa %>%
  addLegend(
    position = "bottomright",
    pal      = paleta_tempo,
    values   = trechos_no_mapa$ORDEM,
    title    = paste0("Ordem do voo (", tail_number, ")"),
    opacity  = 0.8
  )

list(
  tabela = trajetos,
  grafico = mapa
)
}

```

Aplicação da função

```

resultado <- analisa_aeronave(
  tail_number = "N431WN",
  arquivo     = "flights.csv.zip"
)

```

A tabela retornada contém todas as colunas originais do arquivo de vôos, além das colunas criadas para o mapa (data, coordenadas, velocidade média e ordem cronológica). Abaixo, as primeiras linhas, apresentadas apenas com as colunas mais informativas para conferência da ordenação:

```

resultado$tabela %>%
  select(
    DATA, SCHEDULED_DEPARTURE, AIRLINE, FLIGHT_NUMBER,
    ORIGIN_AIRPORT, DESTINATION_AIRPORT, DISTANCE, AIR_TIME, VELOCIDADE_MEDIA
  ) %>%
  head(10) %>%
  knitr::kable(digits = 1)

```

DATA	SCHED- ULED_DE- PARTURE	AIR- LINE	FLIGHT_ BER	ORI- NUM-AIR- PORT	DESTINA- TION_AIR- PORT	DIS- TANCE	AIR_ TIME	VELOCI- DADE_ME- DIA
2015-01-01	1035	WN	1762	DCA	HOU	1218	195	374.8
2015-01-01	1305	WN	3564	HOU	BOS	1609	182	530.4
2015-01-01	1755	WN	3564	BOS	BWI	369	74	299.2
2015-01-01	2020	WN	3564	BWI	MCI	967	176	329.7
2015-01-02	700	WN	1793	MCI	LAX	1363	180	454.3
2015-01-02	905	WN	1793	LAX	LAS	236	44	321.8
2015-01-02	1040	WN	1793	LAS	ICT	986	121	488.9
2015-01-02	1535	WN	1793	ICT	DAL	333	64	312.2
2015-01-02	1720	WN	1793	DAL	MDW	793	97	490.5
2015-01-02	2005	WN	3144	MDW	ATL	591	86	412.3

```
dim(resultado$tabela)
```

```
[1] 1938 43
```

Um resumo do ano da aeronave, usado como leitura de apoio ao mapa:

```
resultado$tabela %>%
  summarise(
    voos = n(),
    aeroportos = n_distinct(DESTINATION_AIRPORT),
    distancia_total = sum(DISTANCE, na.rm = TRUE),
    velocidade_media = mean(VELOCIDADE_MEDIA[is.finite(VELOCIDADE_MEDIA)])
  ) %>%
  knitr::kable(digits = 1)
```

voos	aeroportos	distancia_total	velocidade_media
1938	130	1484118	418.2

Mapa do trajeto

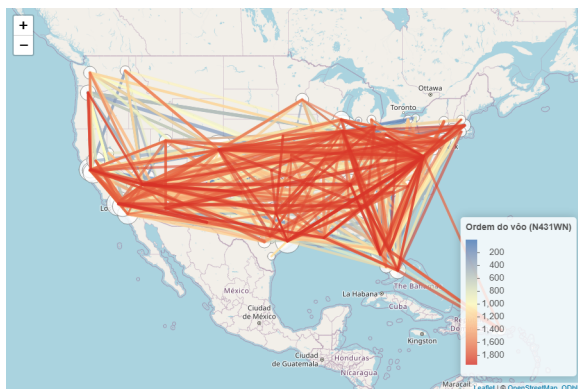
O objeto retornado em `grafico` é um HTML widget (`leaflet`). Para que ele apareça no PDF, o widget é salvo em HTML e convertido em imagem com `webshot2`.

```
chromote::set_chrome_args(c(
  chromote::default_chrome_args(),
  "--no-sandbox", "--disable-dev-shm-usage", "--disable-gpu"
))

saveWidget(resultado$grafico, "mapa_aeronave.html", selfcontained = TRUE)

invisible(webshot2::webshot(
  "mapa_aeronave.html",
  file = "mapa_aeronave.png",
  delay = 5,
  vwidth = 900,
  vheight = 600
))

knitr::include_graphics("mapa_aeronave.png")
```



No mapa, cada linha é um trecho voado pela aeronave. A cor da linha indica a posição do voo na sequência do ano (do azul, no início do ano, ao vermelho, no fim), de modo que o trajeto é lido linearmente no tempo. A espessura da linha é proporcional à velocidade média do trecho. Os círculos marcam os aeroportos visitados, com raio proporcional ao número de pousos da aeronave em cada um deles.

Parte 2

Os nomes das variáveis foram escolhidos de forma descritiva (`trajetos`, `trechos_no_mapa`, `aeroportos_visitados`, `VELOCIDADE_MEDIA`) e os comentários da função registram apenas as decisões que não estão evidentes no próprio código, como os critérios de exclusão de trechos e a construção das escalas de cor e espessura do mapa.

```
Sys.setenv(TZ = "America/Sao_Paulo")
Sys.time()
```

```
[1] "2026-09-16 18:56:50 -03"
```